# The Oracle

*The Oracle at Delphi was the most authoritative voice in the ancient world. Kings and generals sought its guidance before every major decision. The oracle always answered. The answers were always true. And they were almost always misread.*

---

*If you can't see understanding, how do you know it's there?*

I was once told it made me look bad that I hadn't posted anything in the demo channel.

My direct lead pointed it out. He wasn't seeing my contribution and was questioning whether I was contributing at all.

What I had been working on was a package-private, domain-driven setup for a Java service: clean encapsulation, limited visibility of methods and classes, as little side effect as possible. The kind of structural decision that makes everything built on top of it easier, safer, and more maintainable. The bedrock that lets features ship reliably without unexpected interactions. The foundation: nothing shippable, nothing to demo.

I nodded and ignored it. The measurement seemed wrong and I knew the work was right. My role was eliminated shortly after. I've never been able to fully separate the two. When a metric doesn't see someone's work, the people reading the metric don't see it either.

What did I do after being told? I pushed back. I didn't start posting in the demo channel. I informed the team about what I'd been told, questioned it openly, and named it as a bad metric, in the same category as review request count, lines of code, or tokens burned. Metrics that measure activity rather than value, and that get gamed the moment they become targets.

The metric couldn't see it. And when I think about what might have changed the outcome, it wasn't doing different work. It was being asked a different question. Not "why aren't you in the demo channel" but "what are you working on." The first has a measurable answer. The second requires a conversation.

The metric was visibility. The work was invisible. The oracle gave a true signal, and nobody

thought to question the reading. Questioning it is a more useful act than changing your behavior to satisfy it.

Teams have always rewarded the visible over the valuable. The feature over the refactor. The new thing over the stable thing. The demo over the test suite.

But AI makes it catastrophic. Because now the demo channel can be full every day without much effort. Ship a feature, open a review request, close a ticket. The numbers look incredible. And the invisible work, meaning the quality, the structure, the foundation, gets starved of attention because it doesn't register anywhere that matters.

The person doing the invisible work gets told it makes them look bad. Or loses their role. Or quietly stops doing the work that nobody can see.

---

We used to confuse effort with value. AI lets us confuse output with value.

DORA's 2024 research on generative AI[1] put numbers on this. For every 25% increase in AI adoption, local process metrics improved: code quality, documentation, speed of code reviews. The same increase correlated with a 1.5% drop in overall delivery throughput and a 7.2% drop in delivery stability, and developers reported higher flow and personal satisfaction while spending less time on what DORA, that year, categorized as valuable work.

These were correlations, and DORA was careful to say so. The explanation it offered was a hypothesis, not a finding: AI lets a team produce far more code in the same time, changesets grow, and bigger changes have always been slower to land and quicker to break. The time saved got lost downstream: in toil, in reviews that couldn't keep pace, in merges too large to be safe. The oracle measured the right things at the wrong level.

The dividend arrives at the task level, the system can't absorb it, and that gap is what this book is about. DORA's 2024 numbers said throughput fell, not rose, the wrong direction for anyone who already believed implementation was abundant. DORA's 2025 report is the flip: throughput and time on valuable work both reversed positive.[2] What didn't reverse is instability, still climbing on the far side of that flip, and the difference is everything upstream

[1]DORA, *Impact of Generative AI in Software Development* (dora.dev/research/ai/gen-ai-report), the generative-AI companion to the *Accelerate State of DevOps Report 2024*: a 1.5% reduction in delivery throughput and a 7.2% reduction in delivery stability for every 25% increase in AI adoption, estimated associations, not causal claims. DORA, *State of AI-assisted Software Development* (dora.dev, September 2025): the throughput relationship reversed, with AI adoption now associated with increased throughput, improved product performance, and more time on valuable work, while delivery instability continued to increase.

[2]DORA, *Impact of Generative AI in Software Development* (dora.dev/research/ai/gen-ai-report), the generative-AI companion to the *Accelerate State of DevOps Report 2024*: a 1.5% reduction in delivery throughput and a 7.2% reduction in delivery stability for every 25% increase in AI adoption, estimated associations, not causal claims. DORA, *State of AI-assisted Software Development* (dora.dev, September 2025): the throughput relationship reversed, with AI adoption now associated with increased throughput, improved product performance, and more time on valuable work, while delivery instability continued to increase.

of the agent: the thin spec, the wrong thing built quickly, the rework nobody counts. METR supplies a different kind of evidence, not a dividend that failed to land, but one that wasn't there: experienced developers ran 19% slower on repositories they knew well and came away certain they'd gotten faster,[3] the same failure to collect showing up as perception instead of throughput, at the scale of one developer instead of a team. A throughput gain a team cannot land safely is not a dividend it has cashed, and until the process lets it cash, the room costs principal. Appendix A's quarter is the test, and it names the measures. Flat after a quarter of full sessions, and the premise doesn't hold for that team.

---

One more reading, and it's the one I like least. The evidence I have for implementation getting cheap is my own: an engine I could not have written, a refactor that went from two days to two hours, a pace the team could not absorb. All of it self-reported, and the METR trial in this chapter is precisely a finding that self-reports about AI speed are unreliable, in the direction I would want them to be wrong. Sixteen developers believed they were 20% faster and were 19% slower. I have no reason to think I'm exempt. So I should stop making the claim I can't support. Whether AI made me faster at finishing work is exactly the thing METR says I am the worst available witness to, and I am not going to argue my way around a study I put in this chapter myself.

That was never the claim this book needs. What it needs is smaller and harder to dispute: producing something that looks finished, and that somebody now has to read, got radically cheaper. That one leaves marks outside my head. Review requests open at once. A queue that stopped draining. A feature built across a handful of evenings that would have taken a month of them. None of it depends on my sense of how fast I was working.

METR's developers were slower and certain they were faster, and the reason that illusion is so convincing is that the output really was arriving. Their finding and mine are the same finding read from opposite ends. What got cheap was the artifact, not the outcome, and a team's process is calibrated to the rate at which work arrives needing attention rather than the rate at which it gets finished. Only the first one moved. That gap is the subject of this book.

---

The metrics we use to measure engineering output were wrong before AI. AI just made them easier to game and faster to break.

Lines of code. Story points. Number of review requests. Tokens burned. Each of these measures

[3]METR, *Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity* (metr.org, July 2025; arXiv:2507.09089). Sixteen experienced developers were 19% slower with AI on repositories they knew well, yet believed it had made them roughly 20% faster.

something real in isolation. Each becomes meaningless the moment it becomes a target.

Goodhart's Law[4]: when a measure becomes a target, it ceases to be a good measure. The metric stops describing the thing that matters and starts describing the behavior the metric incentivizes.

An AI power user can inflate every one of those metrics simultaneously. Lines of code go up. The agent writes more than a human would. Story points get closed faster. The agent implements tickets in hours. Review requests multiply. Why open one when five will do. Tokens burned increase, and that's just the cost of running the agent.

And the dashboard looks great. Leadership sees throughput.

The review queue is growing underneath it. The specs are getting thinner because there's no time to write them properly when the agent is already running. The codebase is accumulating decisions that made sense in isolation and don't fit together. The team is heads down, individually productive, collectively drifting.

---

These are all activity metrics, counted per person, a choice the numbers never required.

They were questionable even then. But individuals were the unit of delivery: one engineer owned a feature end to end, designing it, building it, testing it, shipping it, and measuring the individual at least pointed at the output.

In a team-delivered, agent-executed workflow, the individual is no longer the right unit. The agent handles implementation. The value comes from what the team understood and agreed on before the agent ran: the spec, the shared context, the collective judgment about what to build and why. None of that shows up in an individual's metrics.

Worse, individual metrics actively work against shared understanding. If story points closed is how an engineer gets recognized, the incentive is to close tickets, not to spend time in a spec session building context that benefits the whole team. If review requests opened is the signal, the incentive is to keep the agent running, not to slow down and question whether the spec is right.

The usual response when this becomes visible: add another skill to the agent. Improve the prompt. Use a better model. Find a way to make the AI produce better output with less review.

The output isn't the problem. The process that produces it is. Adding capability to the agent

[4]After the economist Charles Goodhart (1975); the familiar phrasing is Marilyn Strathern's (1997): "When a measure becomes a target, it ceases to be a good measure."

doesn't fix a broken spec; it produces better-written code against the wrong intent, faster. If that counts as a fix, the incentive has been made more powerful, not less.

---

The metric that actually matters is predictability.

Predictability, not velocity, not throughput, not any measure of how much output the team is producing.

Can the team tell a stakeholder when something will be done, and then actually do it by then?

That's the signal that the process is working. That the specs are good enough to commit from. That the team understands the work before it starts. That surprises are rare because the thinking happened upfront.

Predictability doesn't require the work to be foreseeable. In domains where answers only show up by building, the team changes what it promises, not whether it keeps promises: by Thursday we'll know whether approach A holds, instead of the feature ships Thursday. Shaping the commitment to what the team can actually know is itself understanding, made visible to the people outside the room.

A team with high velocity and low predictability is guessing fast. A team with lower velocity and high predictability has shared understanding. The second team is more valuable, more trustworthy, and more capable of scaling, because when capacity is added to a predictable process, the predictability holds.

When AI is added to a broken process, the breakage accelerates.

Predictability is also the metric that can't be easily gamed. Either it was called right or it wasn't. No amount of demo channel activity changes whether the thing shipped when the team said it would. No number of tokens burned makes a forecast accurate.

And predictability is a team metric, not an individual one. One person can't be predictable alone. Predictability requires shared understanding of the work, shared ownership of the sizing, shared commitment to what goes through the gate. It requires the Spec Session, or something like it: a moment where the team agrees on what is being built and how big it is, while the work is still on the page.

No points, no poker. Estimation was how teams committed when implementation capacity was the constraint, and the constraint moved. What sizing looks like instead is in the working template at the back. What the room costs, honestly counted, is the appendix beside it.

None of this makes predictability immune to gaming. A team can pad the estimate, promise six weeks for work that takes three, and hit the date every time. But that only holds as long as

nobody outside the team asks why three-week features take six, and stakeholders eventually do. Sandbagging predictability takes the whole team agreeing to the same lie and holding it quarter after quarter. Inflating throughput takes one person and an agent for an afternoon.

Teams have always known that upfront alignment produces better estimates, and that's not a new idea. What's new is that AI makes the alternative, skipping the alignment, running the agent, seeing what happens, feel so productive in the moment that the incentive to do the upfront work disappears.

The demo channel fills up. The forecast gets noisier. From the outside, everything looks great.

---

Output is easy to buy now. A thousand tokens spent implementing the wrong thing creates exactly the same value as a thousand hours spent implementing the wrong thing: none.

The first team looks unstoppable on paper: four times the review volume of a year ago, an active demo channel, a healthy burndown chart. Underneath, reviews are backing up, commitments slip, stakeholders have stopped trusting the estimates, and rework is quietly eating the gains.

The second team ships less visibly, but when they say something will be ready on Thursday, it's ready on Thursday. Reviews are fast because the specs were tight. Stakeholders plan around their commitments.

The first team is optimizing for output. The second team is optimizing for understanding.

From a dashboard, the first team looks more productive. From a planning meeting, the second team is more valuable.

---

Reliable forecasts tell a manager something deeper than delivery dates. They say the team shares a model of the system, the problem, and the solution. The alignment was real, and the forecast is the evidence.

The thing to measure isn't how much the team is producing.

It's whether the team can say what they'll produce, and when, and whether they're right.

Output is abundant now, but shared understanding isn't. The teams that turn understanding into reliable commitments are the ones creating value.

Everything else is just numbers getting easier to game.

Predictability shows that the team understands the work. What it doesn't show is how that understanding gets built.